

# 작은 세상 경제 시뮬레이션

웹 기반 경제 시뮬레이터 — 주식 시장부터 시작해 점진적으로 확장하는 살아있는 가상 세계 구현 계획서

|       |                                       |
|-------|---------------------------------------|
| 프로젝트  | 작은 세상 (Small World) Economy Simulator |
| 시작 모듈 | Phase 1 — 주식 시장 (Stock Market)        |
| 구현 방식 | 단계적 확장 (Phased Incremental Build)     |
| AI 연동 | Claude API → 실시간 뉴스 생성 → 주가 반영        |
| 최종 목표 | 완전한 가상 경제 생태계 — 6개 레이어 완성             |

이 문서는 6단계로 확장되는 경제 시뮬레이터의 전체 아키텍처와 단계별 구현 계획을 담고 있습니다.

## 1. 프로젝트 개요

작은 세상(Small World)은 브라우저에서 동작하는 웹 기반 경제 시뮬레이터입니다. 실제 경제 메커니즘을 모사한 6개 레이어가 유기적으로 연결되어, 플레이어가 주식을 거래하고 기업을 운영하는 살아있는 가상 세계를 만드는 것이 목표입니다.

### 핵심 설계 원칙

|                                                   |                                             |
|---------------------------------------------------|---------------------------------------------|
| 점진적 확장 레이어가 안정화된 후 다음 레이어 추가. 복잡도 폭발 방지           | 인과관계 우선 모든 경제 요소가 실제 원인-결과 관계로 연결. 단순 랜덤 아님 |
| AI 뉴스 드라이버 Claude API가 뉴스를 생성하고 그것이 시장에 충격을 주는 구조 | 플레이어 참여 단순 관람이 아닌 거래·투자 등 능동적 참여 가능         |

## 2. 전체 경제 시스템 아키텍처

6개 경제 레이어가 아래에서 위로 쌓이는 구조. 하위 레이어가 완성되어야 상위 레이어가 의미를 가집니다.

|         |                                                            |         |
|---------|------------------------------------------------------------|---------|
| Phase 1 | <b>주식 시장</b><br>기업 종목 · 오더북 · 가격 결정 · AI 뉴스 · 차트 · 포트폴리오   | 지금 구현   |
| Phase 2 | <b>통화 &amp; 중앙은행</b><br>기준금리 · 인플레이션 · 통화량 · 양적완화 이벤트      | Phase 2 |
| Phase 3 | <b>기업 &amp; 실물경제</b><br>생산 · 매출 · 실적발표 · M&A · 파산 · 산업 공급망 | Phase 3 |
| Phase 4 | <b>노동 &amp; 시민</b><br>NPC 고용 · 임금 · 소비 · 저축 · 실업률          | Phase 4 |
| Phase 5 | <b>정부 &amp; 세금</b><br>세율 · 재정정책 · 규제 · 복지 · 국채 발행          | Phase 5 |
| Phase 6 | <b>국제 무역</b><br>환율 · 수출입 · 관세 · 무역협정 · 글로벌 충격              | Phase 6 |

### 레이어 간 주요 인과관계

| 연결 방향     | 영향 경로                 | 예시                          |
|-----------|-----------------------|-----------------------------|
| 주식 ← 중앙은행 | 금리 인상 → 주가 하락 압력      | 기준금리 0.5%p 인상 시 전 종목 평균 -3% |
| 주식 ← 기업실적 | 어닝 서프라이즈 → 해당 종목 급등   | EPS 예상치 초과 시 +8~15% 충격      |
| 기업 ← 노동시장 | 임금 상승 → 원가 증가 → 이익 감소 | 실업률 3% 이하 구간에서 인건비 압박       |

|           |                       |                    |
|-----------|-----------------------|--------------------|
| 전체 ← 정부정책 | 세율·규제 변화 → 업종별 차별 영향  | 방산 예산 증가 → AXIS 수혜 |
| 전체 ← 무역   | 환율 변동 → 수출기업 실적 직접 반영 | 원화 강세 → 수출기업 실적 악화 |

### 3. Phase 1 — 주식 시장 상세 설계

#### 3-1. 가상 기업 목록 (8개 종목)

| 티커   | 기업명    | 섹터     | 초기가(원)  | 변동성    | 특이사항                           |
|------|--------|--------|---------|--------|--------------------------------|
| NOVA | 노바시스템즈 | 반도체/AI | 87,400  | High   | AI 반도체 수혜.<br>빅테크 계약 이벤트<br>빈번 |
| ARCO | 아르코에너지 | 정유/신재생 | 31,600  | Med    | 유가 민감. 신재생<br>정책에 따라 극단<br>등락  |
| BIOM | 바이오메드  | 신약/임상  | 126,500 | V-High | FDA 임상<br>결과·특허로 극단<br>변동 가능   |
| TREK | 트렉물류   | 물류/배송  | 44,800  | Low    | 경기 민감. 소비<br>둔화 시 실적 직격탄       |
| MIRA | 미라금융   | 투자은행   | 68,200  | Low    | 금리 환경 민감.<br>중앙은행 이벤트<br>영향 큼  |
| VOLT | 볼트모터스  | 전기차    | 193,000 | V-High | 보조금·경쟁사<br>동향으로 ±20%<br>가능     |
| GAIA | 가이아푸드  | 식품     | 28,500  | Low    | 방어주 성격.<br>경기침체기 상대적<br>안전     |
| AXIS | 엑시스디펜스 | 방산/사이버 | 53,700  | Med    | 지정학 리스크·국방<br>예산 증액에 반응        |

#### 3-2. 가격 결정 메커니즘

주가는 세 가지 힘의 합산: 기본 랜덤 워크 + AI 뉴스 충격 + 잔류 모멘텀 감쇠.

##### 가격 업데이트 공식 (2초 틱마다 적용)

```
newPrice = price x (1 + drift + momentumEffect)

drift = randomWalk(+/-vol) // 종목별 변동성 계수
momentumEffect = momentum x 0.30 // 누적 충격의 30% 매 틱 반영
momentum = momentum x 0.55 // 매 틱마다 45% 감쇠 (자연 소멸)

[뉴스 충격 적용 시 — applyShock(ticker, impact)]
price += price x impact x 0.55 // 즉시 55% 반영
momentum += impact x 0.45 // 잔류 모멘텀 누적 (45%)
volume += abs(impact) x 3000 // 거래량 스파이크 발생
```

3-3. AI 뉴스 생성 시스템 — 핵심 모듈

Claude API가 현재 시장 상황을 인식하고 현실적인 금융 뉴스를 자동 생성합니다. 생성된 뉴스는 즉시 파싱되어 해당 종목 주가에 충격으로 반영됩니다.

|         |                                             |
|---------|---------------------------------------------|
| 트리거     | 28초 자동 주기 또는 사용자 수동 요청                      |
| 컨텍스트 주입 | 8개 종목 현재가·등락률·시장 분위기를 프롬프트에 포함              |
| AI 생성   | Claude가 뉴스 제목·본문·영향 종목·충격 강도·심각도를 JSON으로 반환 |
| 파싱      | JSON 파싱 후 affected 배열에서 ticker·impact 추출    |
| 가격 반영   | applyShock() 호출 → 즉시 가격 점프 + 잔류 모멘텀 주입      |
| UI 업데이트 | 뉴스 피드에 카테고리 배지·심각도 배지·종목 충격을 표시             |

API 응답 JSON 스키마

```
{
  "headline": "뉴스 제목 (25자 이내)",
  "body": "본문 2문장. 구체적이고 현실적으로.",
  "category": "실적 | 인수합병 | 규제 | 거시경제 | 스캔들 | 신제품 | 계약 | 사고",
  "affected": [{"ticker": "NOVA", "impact": 12.5}],
  "severity": "low | medium | high | extreme"
}
```

impact 범위: extreme +/-15~25 / high +/-8~15 / medium +/-3~8 / low +/-1~3

3-4. AI 시장 참여자 (봇) — 유동성 공급

| 봇 유형  | 전략                      | 역할            |
|-------|-------------------------|---------------|
| 트렌드 봇 | 상승 추세 감지 시 매수 / 하락 시 매도 | 추세 강화, 모멘텀 연장 |
| 역추세 봇 | 급락 시 저점 매수 / 급등 시 차익 실현 | 가격 수렴, 변동성 완화 |
| 인덱스 봇 | 시장 지수 대비 괴리 발생 시 조정 매매  | 종목 간 가격 균형 유지 |
| 뉴스 봇  | 뉴스 이벤트 직후 충격 방향으로 진입    | 뉴스 임팩트 증폭     |
| 랜덤 봇  | 순수 랜덤 매매                | 시장 유동성 기본 공급  |

## 4. 파일 구조 및 모듈 설계

### Phase 1 프로젝트 파일 트리

```
small-world/
+-- src/
| +-- engine/ # 시뮬레이션 코어 (UI 의존성 없음)
| | +-- marketEngine.js # 가격 결정 · 틱 루프 · applyShock()
| | +-- orderBook.js # 오더북 매칭 엔진
| | +-- bots.js # AI 시장 참여자 (5가지 전략)
| | +-- newsEngine.js # Claude API 연동 · JSON 파싱
| | +-- companies.js # 가상 기업 초기 데이터 정의
| |
| +-- store/
| | +-- worldStore.js # Zustand 전역 상태 관리
| |
| +-- components/
| | +-- layout/
| | | +-- Header.jsx # 시장 지수 · 뉴스 생성 버튼
| | | +-- Layout.jsx # 3컬럼 그리드 래퍼
| | +-- market/
| | | +-- Watchlist.jsx # 종목 목록 패널
| | | +-- StockHeader.jsx # 선택 종목 현재가·등락률 바
| | | +-- PriceChart.jsx # Area 차트 (Recharts 기반)
| | | +-- VolumeChart.jsx # 거래량 Bar 차트
| | +-- news/
| | | +-- NewsFeed.jsx # 뉴스 피드 패널
| | +-- trade/
| | +-- OrderPanel.jsx # 매수/매도 주문 입력
| | +-- PortfolioSummary.jsx # 총자산 · 손익 요약
| | +-- HoldingsList.jsx # 보유 종목 목록
| |
| +-- hooks/
| | +-- useMarket.js # 가격 틱 구독 훅
| | +-- useNews.js # 뉴스 인터벌 관리 훅
| +-- App.jsx
+-- package.json
+-- vite.config.js
```

### 핵심 모듈 인터페이스 정의

|                 |                                                                        |
|-----------------|------------------------------------------------------------------------|
| marketEngine.js | tick()으로 전 종목 가격 업데이트. applyShock(ticker, impact)로 뉴스 충격 반영            |
| orderBook.js    | placeOrder(type, ticker, qty)로 매수/매도. 체결 시 holdings 자동 업데이트            |
| newsEngine.js   | generate()로 Claude API 호출. JSON 파싱 후 {headline, affected, severity} 반환 |
| bots.js         | start() / stop()으로 봇 루프 제어. 봇별 전략 함수가 placeOrder() 호출                  |
| worldStore.js   | stocks, history, portfolio, news 전역 상태. Zustand로 리렌더 최소화               |

## 5. 기술 스택

| 라이브러리 / 도구   | 버전       | 용도                                |
|--------------|----------|-----------------------------------|
| React        | 18       | UI 프레임워크. 훅 기반 상태 관리              |
| Vite         | 5+       | 빠른 개발 서버 & 빌드 도구                  |
| Recharts     | 2.x      | 주가 Area 차트 + 거래량 Bar 차트           |
| Zustand      | 4.x      | 경량 전역 상태 관리. Redux 대비 보일러플레이트 최소화 |
| Claude API   | Sonnet 4 | 뉴스 생성 AI. 현재 시장 상황 주입 후 JSON 수신   |
| Tailwind CSS | 3.x      | 유틸리티 클래스 기반 스타일링                  |
| localStorage | 브라우저 내장  | 포트폴리오 · 설정 퍼시스턴스                  |
| Web Worker   | 브라우저 내장  | 틱 루프를 메인 스레드와 분리 (Phase 1 후반 적용)  |

## 6. 개발 로드맵 & 타임라인

| 단계        | 작업명       | 기간 | 주요 내용                                  |
|-----------|-----------|----|----------------------------------------|
| Phase 1-A | 프로젝트 셋업   | 1일 | Vite + React + Zustand 초기화, 기업 데이터 정의  |
| Phase 1-B | 가격 엔진     | 2일 | marketEngine.js 랜덤 워크, 2초 틱 루프 구현      |
| Phase 1-C | 차트 UI     | 2일 | Recharts AreaChart + BarChart + 종목 리스트 |
| Phase 1-D | AI 뉴스 시스템 | 2일 | newsEngine.js, Claude API 연동, 충격 반영 로직 |
| Phase 1-E | 거래 시스템    | 1일 | 매수/매도 주문, 포트폴리오, 손익 계산                 |

|           |           |    |                              |
|-----------|-----------|----|------------------------------|
| Phase 1-F | AI 봇      | 2일 | 5가지 봇 전략 구현, 유동성 공급 검증       |
| Phase 1-G | 마무리·폴리싱   | 1일 | 버그 픽스, 성능 최적화, UI 정리         |
| Phase 2   | 통화 & 중앙은행 | 1주 | 금리 시스템, 인플레이션 모델, Phase 1 연동 |
| Phase 3   | 기업 & 실물경제 | 2주 | 생산/매출 모델, 분기 실적 발표, 공급망      |
| Phase 4   | 노동 & 시민   | 2주 | NPC 에이전트 시스템, 소비 모델          |
| Phase 5   | 정부 & 세금   | 1주 | 재정정책, 세율, 규제 이벤트             |
| Phase 6   | 국제 무역     | 2주 | 환율, 수출입, 글로벌 충격 연동           |



## 7. Phase 2~6 예고 설계

### Phase 2 통화 & 중앙은행

Phase 2

- 중앙은행 위원회가 주기적으로 기준금리 결정 (AI 시나리오 생성)
- 금리 인상 → 주식 밸류에이션 압박 → 성장주 하락 우선
- 인플레이션 지수(CPI) 도입: 상승 시 금리 인상 자동 트리거
- 양적완화/긴축 이벤트: 시장 전반 유동성 변화 반영

### Phase 3 기업 & 실물경제

Phase 3

- 기업별 분기 실적 발표 사이클 (AI가 EPS 수치 생성)
- 산업별 공급망: 반도체 부족 → NOVA 원가 상승 → 영업이익 압박
- M&A; 이벤트: 인수 대상 기업 +20~40% 급등 시뮬레이션
- 기업 파산 이벤트: 상장폐지 리스크 및 섹터 전파 효과

### Phase 4 노동 & 시민

Phase 4

- NPC 시민 수천 명 에이전트 시뮬레이션 (계층별 소비 패턴 차이)
- 고용률 → 소비 수준 → 기업 매출에 직접 반영
- 임금 상승 압박 → 기업 영업이익 압박 → 주가 하락 경로
- 저축률 변화 → 은행 예금 → 대출 이자율에 2차 영향

### Phase 5 정부 & 세금

Phase 5

- 법인세율 조정 이벤트 → 기업 순이익에 직접 영향
- 방산/R&D; 예산 증가 → 특정 섹터 수혜 (AXIS, BIOM)
- 환경 규제 강화 → 에너지 섹터 비용 증가 (ARCO)
- 선거 이벤트 → 정책 불확실성 → 시장 변동성 확대

### Phase 6 국제 무역

Phase 6

- 실시간 원/달러 환율 시뮬레이션
- 수출 기업(AXIS, NOVA): 환율 상승 시 수익 개선
- 수입 의존 기업(GAIA): 환율 상승 시 원가 상승
- 관세 이벤트 및 글로벌 경기 지수 연동

## 8. 핵심 데이터 모델

Stock 상태 객체 (Zustand worldStore)

```
{
  ticker : string // 'NOVA'
```

```
name : string // '노바시스템즈'
sector : string // '반도체'
basePrice : number // 초기 기준가 (세션 내 고정)
price : number // 현재가 (2초 틱마다 업데이트)
open : number // 세션 시작가
change : number // 가격 변동폭 (price - open)
changePct : number // 등락률 (%)
volume : number // 누적 거래량
momentum : number // 잔류 뉴스 충격 (0으로 감쇠)
vol : number // 종목별 변동성 계수 (0.01~0.03)
color : string // 차트 대표 색상 (hex)
}
```

NewsItem 객체

```
{
  id : number // timestamp (고유 키)
  headline : string // AI 생성 뉴스 제목
  body : string // AI 생성 뉴스 본문
  category : string // '실적' | '인수합병' | '규제' | ...
  affected : Array // [{ticker: 'NOVA', impact: 12.5}]
  severity : string // 'low' | 'medium' | 'high' | 'extreme'
  catColor : string // 카테고리별 배지 색상 (hex)
  time : string // '14:32' 형식 표시 시각
}
```

구현 시작 순서 요약

|        |                                                 |
|--------|-------------------------------------------------|
| Step 1 | Vite + React + Zustand 프로젝트 초기화                 |
| Step 2 | companies.js — 8개 가상 기업 데이터 정의                  |
| Step 3 | marketEngine.js — 랜덤 워크 틱 루프 구현                 |
| Step 4 | Recharts 차트 UI 연동 (PriceChart + VolumeChart)    |
| Step 5 | newsEngine.js — Claude API 연동 및 applyShock() 구현 |
| Step 6 | OrderPanel + Portfolio — 거래 시스템 완성              |
| Step 7 | bots.js — AI 봇 5종 추가 → Phase 1 완성               |

각 Phase가 완성되면 다음 Phase 상세 계획서를 별도로 작성합니다.